Section 3.3: The Forward Pass

Computing a Prediction Step by Step

AI-IE Class — Lecture Notes

The goal right now is to perform gradient descent on a simple feedforward network. The first step is to compute the **forward pass**, which means computing the network's prediction $\hat{y}$ for a given input $\mathbf{x}$.

1 The Network Architecture

We will work with a **two-layer feedforward network** (one hidden layer) designed to solve XOR:

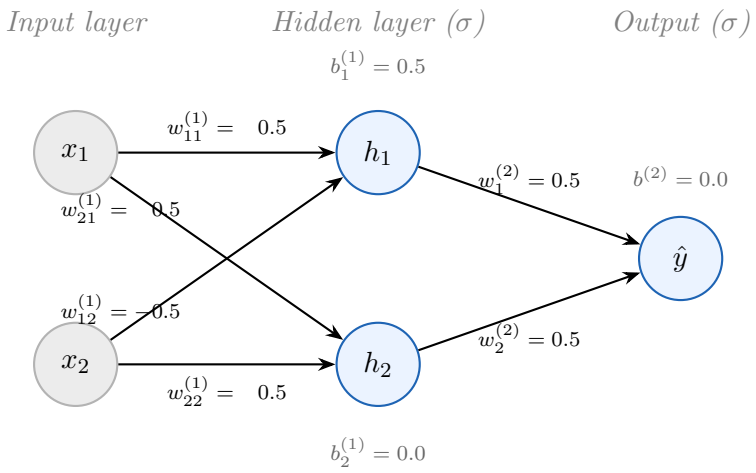


Figure 1: A two-layer feedforward network with one hidden layer.

The network has the following parameters:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 \\ 0.0 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix}, \quad b^{(2)} = 0.0.$$

Row i of $\mathbf{W}^{(1)}$ holds the weights *feeding into* hidden unit i. These weights are **not yet trained** — we will start gradient descent in Section 3.4.

2 Sigmoid Function

In Section 3.1, we used the **step function** to compute the hidden activations h. The step function is rarely used in practice because it is flat everywhere except at the jump, so its derivative is 0 (and undefined at the jump itself). Gradient descent (Section 3.2) moves the weights in the direction given by the gradient. A zero gradient means a small change in a weight (almost) never changes the output, so the network gets no signal telling it which way to move.

We therefore replace it with a smooth "S-shaped" version of the step, the **sigmoid** function. Each neuron in the hidden and output layers uses the sigmoid activation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma(z) \in (0, 1).$$

The computation inside every neuron follows two sub-steps:

1. **Pre-activation** z: a weighted sum of inputs plus bias.
2. **Activation** $h = \sigma(z)$: squeeze z into $(0, 1)$.

Making both sub-steps explicit, every neuron in our network splits into a z node followed by an activation node:

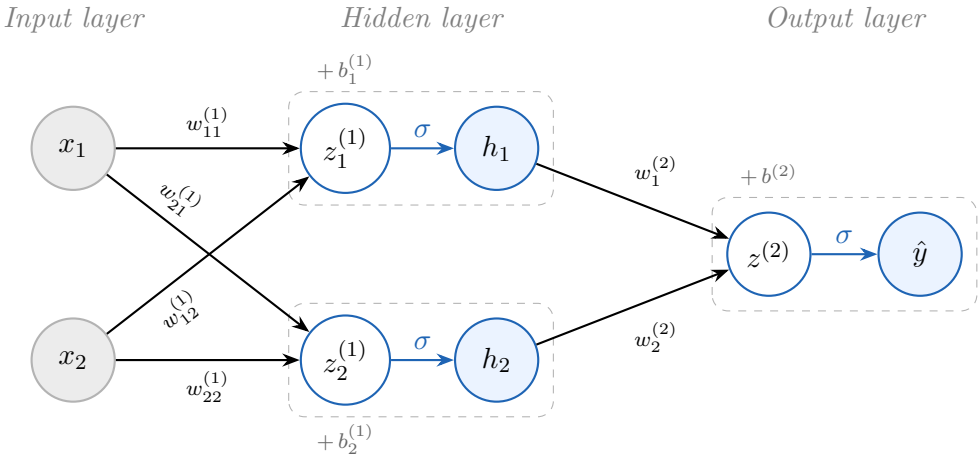


Figure 2: The same network with each neuron (dashed box) split into its two sub-steps. The white node holds the pre-activation z, the weighted sum of the incoming values plus the bias. The blue node holds the activation obtained by applying the sigmoid, $h = \sigma(z)$ (or $\hat{y} = \sigma(z^{(2)})$ at the output).

Computing σ with NumPy. You can evaluate the sigmoid in Python with:

```
import numpy as np

def sigmoid(x):
return 1 / (1 + np.exp(-x))
```

3 The Forward Pass

We trace the network with the XOR input $\mathbf{x} = (x_1, x_2) = (0, 1)$, whose target output is $y = 1$.

Step 1 Input → Hidden pre-activations

Compute the weighted sum entering each hidden neuron:

$$z_1^{(1)} = w_{11}^{(1)}\, x_1 + w_{12}^{(1)}\, x_2 + b_1^{(1)} = (0.5)(0) + (-0.5)(1) + 0.5 = 0.0$$

$$z_2^{(1)} = w_{21}^{(1)}\, x_1 + w_{22}^{(1)}\, x_2 + b_2^{(1)} = (0.5)(0) + (0.5)(1) + 0.0 = 0.5$$

Step 2 Hidden activations

Apply the sigmoid to each pre-activation (use the `sigmoid` function above):

$$h_1 = \sigma\left(z_1^{(1)}\right) = \sigma(0.0) = 0.500$$

$$h_2 = \sigma\left(z_2^{(1)}\right) = \sigma(0.5) = 0.622$$

Step 3 Hidden → Output pre-activation

The output neuron collects the two hidden activations:

$$z^{(2)} = w_1^{(2)}\, h_1 + w_2^{(2)}\, h_2 + b^{(2)} = (0.5)(0.500) + (0.5)(0.622) + 0.0 = 0.250 + 0.311 = 0.561$$

Step 4 Output activation

$$\hat{y} = \sigma\left(z^{(2)}\right) = \sigma(0.561)$$

Using the `sigmoid` function above, `sigmoid(0.561)` gives:

$$\hat{y} \approx 0.637$$

4 Computing the Loss

The target for input $(0, 1)$ is $y = 1$. Using Mean Squared Error:

$$J = (y - \hat{y})^2 = (1 - 0.637)^2 = (0.363)^2 \approx 0.132$$

The network is wrong: it outputs 0.637 when it should output 1. The loss $J = 0.132$ tells us how far off we are.

A perfect prediction ($\hat{y} = 1$) would give $J = 0$. A blind guess of $\hat{y} = 0.5$ would give $J = (0.5)^2 = 0.25$. Our untrained network ($J = 0.132$) happens to be a bit better than random — but only because we are looking at a single input.

5 Summary: What the Forward Pass Gives Us

The network has the following parameters:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 \\ 0.0 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix}, \quad b^{(2)} = 0.0.$$

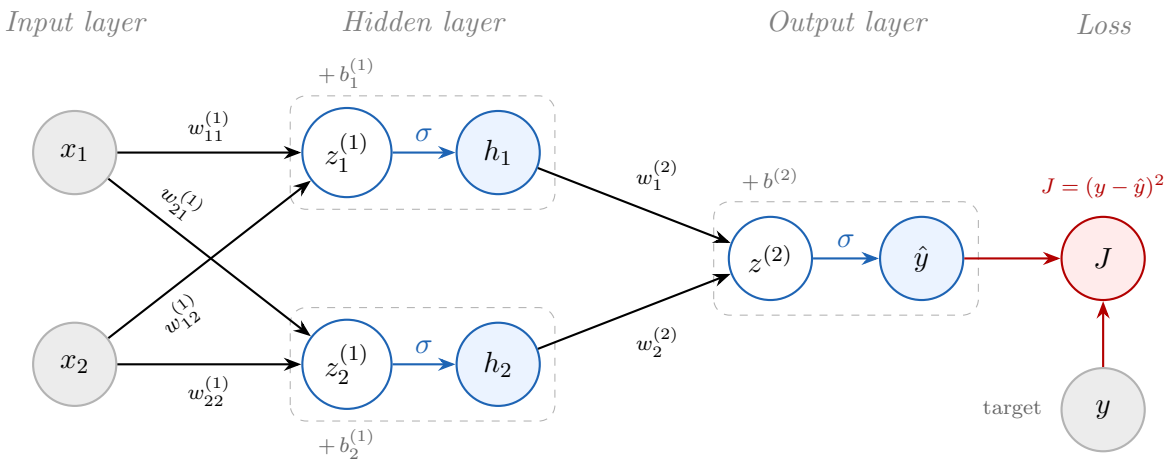


Figure 3: The complete forward pass, from the inputs x_1, x_2 through the pre-activations z and activations h to the prediction $\hat{y}$. The prediction is then compared with the target y to give the loss $J = (y - \hat{y})^2$. Backpropagation (Section 3.4) traverses this same graph in reverse, from J back to every weight.

Quantity	Symbol	Value
Input	$(x_1,\ x_2)$	$(0,\ 1)$
Target	y	1
Hidden pre-activation 1	$z_1^{(1)}$	0.0
Hidden pre-activation 2	$z_2^{(1)}$	0.5
Hidden activation 1	h_1	0.500
Hidden activation 2	h_2	0.622
Output pre-activation	$z^{(2)}$	0.561
Prediction	$\hat{y}$	0.637
Loss	J	0.132

These numbers are the starting point for Section 3.4. Backpropagation will work *backwards* through this same network to compute $\partial J/\partial w$ for every weight — which is exactly what gradient descent needs to take an update step.